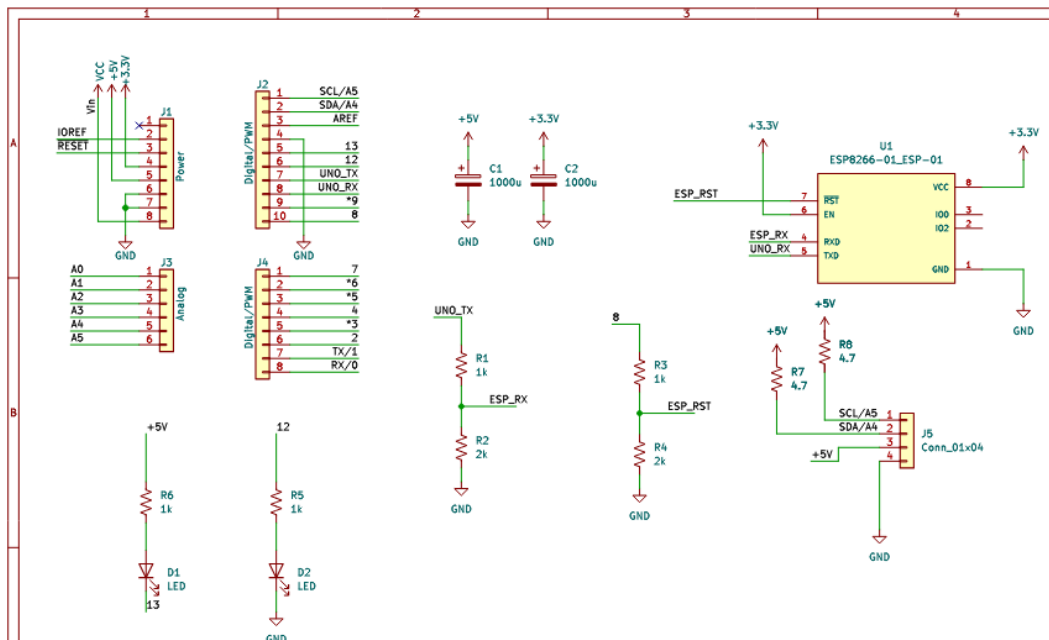


5/16/2025

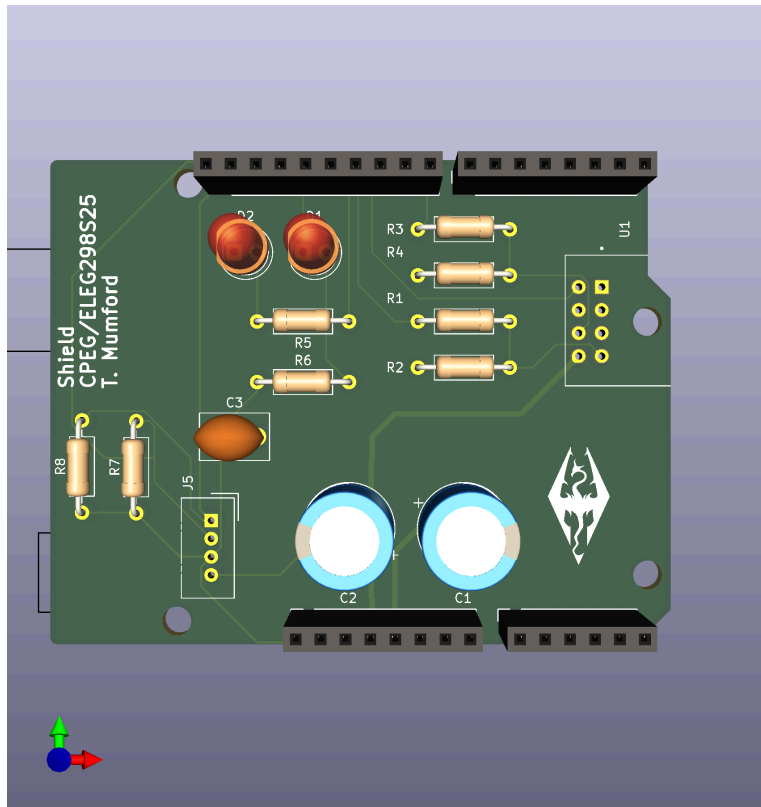
This project helped me get familiar with PCB design, and I also learned how to integrate Grove connectors and keep the circuit small and functional.

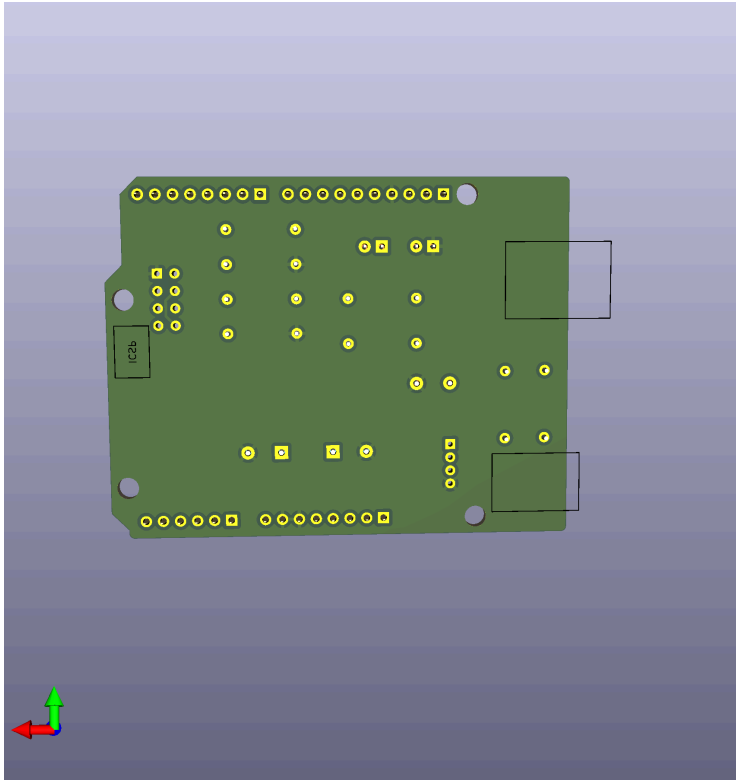
See the attached schematic Png generated in KiCad. It shows all the components like the ESP8266 module, LEDs, resistors, and Grove connector. Everything was kept minimal to keep the board compact.



3. CAM Images (Manufacturing View)

Top and bottom images of the PCB were exported from the KiCad 3D viewer. These show what the finished board would look like. The layout was kept as neat as possible given the limited space.





4. Bill of Materials (BOM)

Part	Value	Qty	Notes
ESP8266-01	WiFi Module	1	Main microcontroller
R1, R3, R5, R6	1k Resistors	4	Used for pull-up / LED current limiting
R2, R4	2k Resistors	2	Divider or biasing
R7, R8	4.7 Ohm Resistors	2	For LEDs
C1, C2	1000uF Capacitors	2	Power filtering
D1, D2	LEDs (Red, Green)	2	Status indicators
Grove Connector	4-pin	1	Sensor input
Misc Headers	-	-	For power and GPIO access

All components were either from the Grove Starter Kit or standard through-hole parts.

5. Arduino Code Description

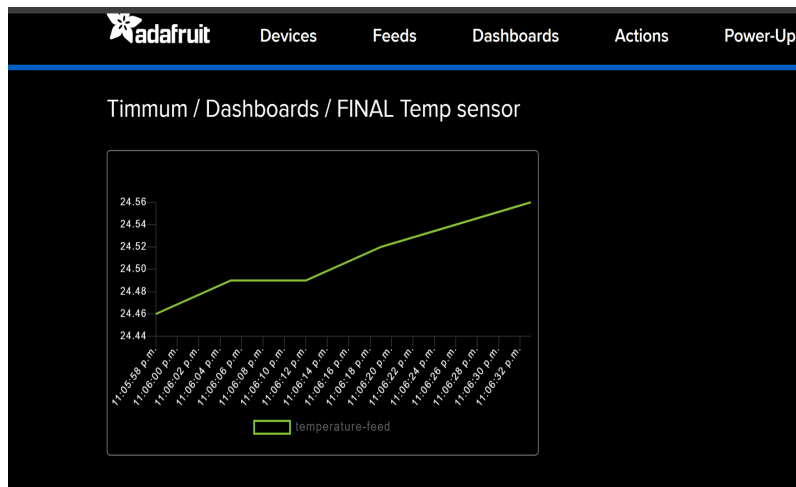
The code was written using the Arduino IDE and uses standard libraries. It reads temperature data from the Grove sensor. If the temp goes above a certain threshold (like 30 degrees Celsius), it turns on the red LED. Otherwise, it stays green. I also added Adafruit IO support so it could upload the temp readings to the cloud.

Libraries used:

- `ESP8266WiFi.h`
- `AdafruitIO_WiFi.h`

6. Adafruit IO Dashboard

A screenshot of the Adafruit IO dashboard is included. It graphs the temperature readings over time so I can look back and see if temps got too high while gaming.



7. Design Improvements

If I were to do this project again, I'd try to redesign it so I could hook it straight to my PC's motherboard for power. The whole reason I made this was so I could keep an eye on temps while gaming, so if I could connect it to something like a USB header or internal 5V rail that'd be cleaner. That way I wouldn't need a separate cable or wall plug.

8. Appendix: Arduino Code

```
#include "Arduino.h"
#include <SoftwareSerial.h>
#include "Grove_Temperature_And_Humidity_Sensor.h"

#define RESET_PIN 8           // pin used to bring ESP-01 module up in UART
mode
#define UART_TX_PIN 11        // UART software serial transmit pin
#define UART_RX_PIN 10        // UART software serial receive pin

#define DHTTYPE DHT20
#define DHTPIN 2
DHT dht(DHTPIN, DHTTYPE);

// LED pins
#define RED_LED_PIN 13
#define GREEN_LED_PIN 12

SoftwareSerial espSerial(UART_RX_PIN, UART_TX_PIN); // Create software
UART to talk to the ESP8266
String IO_USERNAME = "Timmum";
String IO_KEY = "aio_yVXB53vtYaXUfBuhB20TLqv0Pang";
String WIFI_SSID = "UD Devices";
String WIFI_PASS = "";

float num = 1.0;
float lastTemperature = 0.0;
float lastRiseTemp = 0.0;
const float TEMP_TOLERANCE = 0.1;
const float STABLE_LOW = 22.0;
const float STABLE_HIGH = 24.0;
String response;
long int time;

void setup() {
  Serial.begin(115200);
  espSerial.begin(9600);
  pinMode(RESET_PIN, OUTPUT);
  digitalWrite(RESET_PIN, LOW);
  delay(1000);
  digitalWrite(RESET_PIN, HIGH);
```

```

Wire.begin();
dht.begin();

pinMode(RED_LED_PIN, OUTPUT);
pinMode(GREEN_LED_PIN, OUTPUT);

digitalWrite(RED_LED_PIN, LOW);
digitalWrite(GREEN_LED_PIN, LOW);

Serial.println("Setup Complete");

espSend("wifi_ssid=" + WIFI_SSID, 2000, true);
espSend("wifi_pass=" + WIFI_PASS, 2000, true);
espSend("io_user=" + IO_USERNAME, 2000, true);
espSend("io_key=" + IO_KEY, 2000, false);
espSend("setup_io", 3000, true);
espSend("setup_pubfeed=1,temperature-feed", 2000, false);
}

void loop() {
  delay(5000);
  float temp_hum_val[2] = {0};

  if (!dht.readTempAndHumidity(temp_hum_val)) {
    float currentTemp = temp_hum_val[1];
    Serial.print("Current Temperature: ");
    Serial.println(currentTemp);
    float tempDifference = currentTemp - lastTemperature;

    if (tempDifference > TEMP_TOLERANCE) {
      lastRiseTemp = currentTemp;
      digitalWrite(RED_LED_PIN, LOW);
      digitalWrite(GREEN_LED_PIN, HIGH);
      delay(250);
      digitalWrite(GREEN_LED_PIN, LOW);
      delay(250);
    } else if (tempDifference < -TEMP_TOLERANCE && currentTemp <
lastRiseTemp) {
      digitalWrite(GREEN_LED_PIN, LOW);
      digitalWrite(RED_LED_PIN, HIGH);
      delay(250);
      digitalWrite(RED_LED_PIN, LOW);
      delay(250);
    }
  }
}

```

```

    } else if (currentTemp >= STABLE_LOW && currentTemp <= STABLE_HIGH) {
        digitalWrite(GREEN_LED_PIN, LOW);
        digitalWrite(RED_LED_PIN, HIGH);
    }

    response = espSend("send_data=1," + String(currentTemp), 2000, false);
    lastTemperature = currentTemp;
} else {
    Serial.println("Failed to get temperature and humidity value.");
    digitalWrite(RED_LED_PIN, LOW);
    digitalWrite(GREEN_LED_PIN, LOW);
}
}

String espSend(String command, const int timeout, boolean debug) {
    response = "";
    espSerial.println(command);
    long int time = millis();
    while ((time + timeout) > millis()) {
        while (espSerial.available()) {
            char c = espSerial.read();
            response += c;
        }
    }
    response.trim();
    if (debug) {
        Serial.println("Resp: " + response);
    }
    return response;
}

```